

Instituto Tecnológico de Costa Rica
Escuela Ingeniería en Computadores
Curso: CE 1104 Fundamentos de Sistemas
Computacionales

Documentación técnica strangerTEC

Estudiantes:

Mario Camilo Rodríguez Gaitán

Evan Alejandro Umaña Sojo 2026009696

Profesor:

Leonardo Andrés Araya Martínez

Semestre I 2026

0. Índice

0. Índice.....	2
1. Introducción.....	4
1.1. Propósito del documento.....	4
1.2. Descripción del proyecto.....	4
1.3 Objetivos y Alcance.....	4
2. Descripción General del Sistema.....	5
2.1. Perspectiva del Producto.....	5
2.2. Funcionalidades principales.....	5
2.3. Restricciones del sistema, Supuestos y Dependencias.....	5
3. Requisitos del Sistema.....	7
3.1. Requisitos Funcionales.....	7
3.2. Requisitos No Funcionales.....	7
4. Diseño de la Solución.....	8
4.1. Diagrama de Flujo y Secuencia.....	9
4.2. Decisiones de Diseño y Justificación.....	9
5. Implementación y Validación de Calidad.....	11
5.1. Estructura del repositorio.....	11
5.2. Entorno de Desarrollo, Guía de instalación y Ejecución.....	11
5.3. Estrategias y Casos de Prueba.....	12
5.4. Resultados de pruebas y problemas encontrados.....	12
6. Declaración de uso de Inteligencia Artificial.....	14
6.1. Herramientas de IA utilizadas.....	14
6.2. Herramientas de IA utilizadas.....	14

6.3. Declaración de Responsabilidad y Autoría.....	15 7.
Conclusión.....	16 8.
Bibliografía.....	17



1. Introducción

Este documento describe la especificación técnica del sistema de radar 2D desarrollado con Arduino y una interfaz gráfica en Python.

1.1. Propósito del documento

Su propósito es servir como referencia técnica completa del diseño, requisitos e implementación del proyecto, detallando las decisiones tomadas y la justificación de cada una.

1.2. Descripción del proyecto

El proyecto consiste en un sistema de detección y visualización de objetos en tiempo real basado en un sensor ultrasónico montado sobre un servomotor, controlado por un Arduino. El sistema barre un arco midiendo distancias, transmite los datos por puerto serial y los visualiza en una interfaz gráfica tipo radar militar desarrollada en Python con tkinter. La interfaz incluye un sistema de tracking de objetos múltiples con estimación de velocidad y predicción de posición futura.

1.3 Objetivos y Alcance

- Detectar objetos dentro de un rango máximo de 4.5 m en un semicírculo.
- Visualizar en tiempo real la posición de los objetos detectados sobre un display tipo radar polar.
- Implementar un sistema de tracking capaz de seguir múltiples objetos simultáneamente, calculando su velocidad y prediciendo su posición futura.

El sistema cubre desde la adquisición de datos por el sensor ultrasónico hasta la visualización gráfica en pantalla. No incluye almacenamiento persistente de sesiones, autenticación de usuarios ni transmisión de datos por red. La detección se limita a objetos que reflejen adecuadamente el sonido dentro del rango del HC-SR04.

2. Descripción General del Sistema

Esta sección ofrece una visión de alto nivel del sistema: sus características generales, las restricciones bajo las cuales opera y los supuestos que guiaron su diseño.

2.1. Perspectiva del Producto

El sistema opera como una aplicación standalone de escritorio que se comunica con el hardware Arduino a través de un puerto serial USB. No requiere conexión a internet ni servicios externos. La interfaz gráfica corre en cualquier sistema operativo con soporte para Python 3 y tkinter, mientras que el firmware corre exclusivamente sobre Arduino UNO.

2.2. Funcionalidades principales

- Barrido angular automático: el servomotor barre de 0° a 180° y de regreso de forma continua, tomando una medición de distancia en cada grado.
- Transmisión serial eficiente: el Arduino transmite cada par ángulo:distancia separado por coma, permitiendo a Python parsear por token sin depender de saltos de línea.
- Visualización polar en tiempo real: los objetos se representan sobre un semicírculo con anillos de distancia y líneas angulares de referencia, con un trail que se desvanece con el tiempo.
- Línea de sweep sincronizada: la línea giratoria del radar sigue exactamente el ángulo actual del servomotor, leído directamente del serial.
- Tracking de múltiples objetos: el sistema asocia nuevos datapoints a objetos existentes por distancia euclidiana, asignando a cada objeto un color, ID, velocidad en cm/s y predicción de posición futura.
- Panel de estado en tiempo real: muestra lecturas totales, puntos activos, ángulo del sweep y lista de objetos trackeados con sus métricas.

2.3. Restricciones del sistema, Supuestos y Dependencias

Restricciones:

- El sensor HC-SR04 tiene un rango efectivo de 2 cm a 400 cm y un ángulo de detección de aproximadamente 15° , lo que puede generar lecturas falsas en bordes o superficies irregulares.
- La resolución angular es de 1° por paso, limitada por la velocidad del servomotor y el tiempo de medición ultrasónica.
- La comunicación serial opera a 9600 baudios, lo que limita la capacidad de paso máximo de datos. A mayor velocidad de barrido, menor tiempo disponible por medición.
- El timeout de pulseIn está configurado en 26,000 μ s, cubriendo distancias de hasta aproximadamente 440 cm. Los objetos más lejanos no se grafican.

- tkinter no es thread-safe, por lo que toda modificación de la interfaz debe ocurrir en el hilo principal.

Supuestos:

- El Arduino está conectado al puerto COM5 a 9600 baudios.
- El entorno de ejecución tiene Python 3.8 o superior instalado.
- El sistema operativo reconoce el Arduino como dispositivo serial sin drivers adicionales.

Dependencias:

Componente	Versión	Propósito
Python	3.8+	Entorno de ejecución
tkinter	Incluido en Python	Interfaz gráfica
pyserial	3.x	Comunicación serial
Arduino	2.x	Compilación y carga del firmware
Librería Servo.h	Incluida en Arduino IDE	Control del servomotor

3. Requisitos del Sistema

3.1. Requisitos Funcionales

RF-00 1	Barrido continuo	Alta	El sistema debe barrer de 0° a 180° y de regreso de forma continua sin intervención del usuario, con un delay de aproximadamente 5 ms por grado.
RF-00 2	Medición de distancia	Alta	En cada grado del barrido se debe tomar una medición con el sensor HC-SR04 usando pulseIn con timeout de 26,000 μ s.
RF-00 3	Filtrado de lecturas	Media	Solo se transmiten lecturas con distancia mayor a 0 cm y menor a 450 cm. Lecturas fuera de rango se transmiten como 0 para mantener el sweep sincronizado.
RF-04	Transmisión serial	Alta	Cada medición se transmite en formato ángulo:distancia, por el puerto serial a 9600 baudios, incluyendo ángulos sin objeto detectado con distancia 0.
RF-05	Parsing por token	Alta	La aplicación Python debe parsear los datos cortando por coma, sin depender de saltos de línea, procesando cada token (ángulo:distancia) de forma independiente.
RF-06	Visualización polar	Alta	Los objetos detectados deben representarse en coordenadas polares sobre un semicírculo con 5 anillos de distancia y líneas angulares cada 30°.
RF-07	Sweep sincronizado	Baja	La línea de sweep debe seguir el ángulo exacto reportado por el Arduino, no una animación independiente.
RF-08	Tracking multi-objeto	Baja	El sistema debe asociar nuevos puntos a tracks existentes si la distancia euclidiana es menor al máximo de radio de los píxeles, o crear un track nuevo en caso contrario.
RF-09	Métricas de tracking	Baja	Cada objeto trackeado debe mostrar su velocidad en cm/s, posición actual en ángulo y distancia, y una predicción de posición futura proyectada 3 frames más adelante.
RF-10	Eliminación de tracks perdidos	Baja	Un track sin actualizaciones debe eliminarse automáticamente.

3.2. Requisitos No Funcionales

RNF-001	Rastro con fade	Estética de la UI	Cada punto detectado debe permanecer visible desvaneciendo su color verde antes de desaparecer
RNF-002	Interfaz responsiva	Experiencia de Usuario	El radar debe dibujarse correctamente al cambiar el tamaño de la ventana, calculando geometría en tiempo real.
RNF-003	Robustez serial	Validación de casos	El parser debe tolerar tokens malformados, bytes inválidos y desconexiones inesperadas sin crashear la aplicación.
RNF-004	Usabilidad	Experiencia de Usuario	La interfaz debe mostrar claramente el estado de conexión, número de objetos trackeados y métricas en tiempo real en el panel lateral.

4. Diseño de la Solución

La solución se divide en dos componentes independientes que se comunican por serial:

- Firmware (Arduino): responsable del control del servomotor, disparo del sensor ultrasónico y transmisión serial. Opera en un loop determinístico sin interrupciones externas.
- El sistema de tracking implementa un algoritmo de asociación greedy: por cada nuevo datapoint, busca el track existente más cercano dentro de la constante MAX_TRACK_RADIUS. Si lo encuentra lo actualiza, si no crea un track nuevo. La velocidad se estima promediando los deltas de posición de los últimos 5 puntos históricos y multiplicando por FPS para obtener cm/s.
- Aplicación Python: responsable de recibir, parsear y visualizar los datos. Internamente se divide en tres capas:
 - Capa de I/O : hilo serial dedicado (_read_serial) que lee caracteres y encola tokens parseados.
 - Capa de datos : _draw_frame consume la cola, actualiza el estado interno (points, tracks) y calcula métricas.
 - Capa de presentación : dibuja todos los elementos visuales sobre el canvas de tkinter usando tags "base" y "dynamic" para separar elementos estáticos de dinámicos.

4.1. Diagrama de Flujo y Secuencia

4.5. Decisiones de Diseño y Justificación

Lectura serial por carácter en vez de readline()	El Arduino usa Serial.print() sin println(), por lo que nunca se envía un carácter \n. Usar readline() bloquearía el hilo serial indefinidamente esperando un salto de línea que no llega. La solución es leer carácter por carácter y cortar en ,, que sí se envía al final de cada token.
Quiero queue.Queue como puente entre hilos	tkinter no es thread-safe: cualquier llamada a métodos del canvas desde un hilo secundario puede causar crashes silenciosos o condiciones de carrera. La cola permite que el hilo serial solo escriba datos y que el hilo principal de tkinter sea el único que los consuma y modifique el estado visual.

Timeout de pulseIn en lugar del default	El timeout por defecto de pulseIn es 1,000,000 μ s (1 segundo). Con un delay de 5ms por paso, un timeout de 1 segundo hace que cada medición sin eco tarde hasta 1005 ms, enlenteciendo el barrido completo en varios minutos. Con un timeout de 26,000 μ s el peor caso por medición es ~31ms, manteniendo el barrido fluido.
Transmisión de ángulo aunque no haya objeto	Si el Arduino solo transmite cuando hay un objeto detectado, el sweep de Python solo se mueve cuando algo está en rango, quedándose congelado en zonas vacías. Transmitir siempre el ángulo con distancia 0 permite que el sweep sea una representación fiel de la posición real del servomotor en todo momento.
Tags "base" y "dynamic" en el canvas	En vez de mantener referencias a cada elemento dibujado para moverlos o eliminarlos individualmente, se borran todos los elementos de un tag en bloque con <code>canvas.delete("tag")</code> . Los elementos estáticos (anillos, líneas angulares, etiquetas) usan "base" y solo se redibujan en <code>resize</code> . Los elementos dinámicos usan "dynamic" y se redibujan completos en cada frame.
Algoritmo de tracking greedy por distancia euclidiana	Se eligió por su simplicidad de implementación y suficiencia para el caso de uso. Para un radar de corto alcance con pocos objetos simultáneos, la asociación al vecino más cercano funciona correctamente. Un algoritmo de asignación óptima como el Húngaro sería más preciso en escenarios con objetos cruzándose, pero añade complejidad innecesaria para este contexto.
Promedio de últimos 5 deltas para velocidad	Calcular velocidad solo entre el último y penúltimo punto es muy sensible al ruido del sensor, que puede reportar variaciones de varios centímetros entre mediciones consecutivas. Promediar los últimos 5 deltas suaviza el resultado y produce una estimación de velocidad más estable y legible.

5. Implementación y Validación de Calidad

El proyecto está separado con la finalidad de mantener una estructura clara para facilitar su mantenimiento o continuo desarrollo.

5.1. Estructura del repositorio

Se divide en el archivo de la interfaz gráfica, la carpeta del del Arduino y la carpeta multimedia. Se separaron los archivos para hacer claro dónde encontrar cada categoría del proyecto. Se tomaron en cuenta cada transición entre los estados del programa antes de programarlos para asegurarse de evitar cualquier vacío en la lógica.

```
/Radar2D/  
├─ Radar2D/  
│   └─ Radar2D.ino  
├─ ui.py  
├─ docs/  
│   └─ Documentacion_Externa_Arduino.pdf  
├─ media/  
└─ README.md
```

5.2. Entorno de Desarrollo, Guía de instalación y Ejecución

Requisitos de hardware:

- Arduino UNO
- Sensor ultrasónico HC-SR04
- Servomotor estándar (5V)
- Cable USB tipo B

Instalación del firmware:

1. Abrir Radar2D.ino en Arduino IDE 2.x.
2. Seleccionar placa Arduino UNO y el puerto COM correspondiente.
3. Cargar el sketch con el botón Upload.

Ejecución:

1. Conectar el Arduino por USB.
2. Verificar en qué puerto COM aparece

3. Si es distinto de COM5, modificar self.port_var en RadarApp.__init__.
4. Ejecutar python radar_ui.py.
5. Presionar CONNECT en el panel izquierdo.

6. Declaración de uso de Inteligencia Artificial y herramientas

Principio de Transparencia Académica

El uso de herramientas de Inteligencia Artificial en proyectos académicos requiere declaración explícita y responsable. Esta sección documenta de forma honesta cómo, cuándo y para qué se utilizaron herramientas de IA en el desarrollo de este proyecto, siguiendo los principios de integridad académica

6.1. Herramientas de IA utilizadas

Lista todas las herramientas de IA utilizadas durante el desarrollo. Sé específico con el nombre, versión (si aplica) y propósito.

Ejemplo:

Gemini 3 Flash	Modelo de lenguaje (LLM)	Junio 2026	Consulta de parámetros, tipos de retorno.
Claude Sonnet 4.6	Definición de identidad visual	Junio 2026	Creación de una paleta de colores, tipografía, estilo de botones, propuesta de mejora para el diseño de la interfaz y la animación del rastro de los datapoints.
Claude Sonnet 4.6	Troubleshooting	Junio 2026	Asistencia en el ensamblaje y soldadura de los componentes. Profesor de uso de Fusion360 como software de modelado. Debugger en la animación del sweep.

6.2. Descripción del Uso de Inteligencia Artificial

El desarrollo de este proyecto fue asistido por Claude (Anthropic), utilizado como herramienta de apoyo a lo largo de todo el ciclo de desarrollo, desde la depuración del firmware hasta el diseño de la UI.

Ayudó con un problema de lentitud del servomotor, identificando que el timeout por defecto de pulseIn() de 1,000,000 µs bloqueaba el loop por hasta 1 segundo por medición. Identificó el race

condition entre el hilo serial y el hilo de tkinter como causa de que los puntos no se dibujaran hasta desconectar el Arduino cuando no había eco, y proponiendo reducirlo a 26,000 μ s para cubrir el rango real del sensor. El readline() bloqueaba el hilo serial indefinidamente porque el Arduino nunca envía \n, y propuso un rediseño del lector. Identificó el bug en el glow del sweep donde causaba que la línea saltara al ángulo opuesto al llegar a los extremos. Diagnosticó problemas de conexión serial ayudando a verificar cables, timeout y rango del sensor cuando el Serial Monitor aparecía vacío.

6.3. Declaración de Responsabilidad y Autoría

DECLARACIÓN DE AUTORÍA Y RESPONSABILIDAD

Los suscritos, integrantes del equipo de trabajo del proyecto "Radar2D", declaramos que:

1. El diseño, análisis, desarrollo y conclusiones del presente proyecto son resultado de nuestro trabajo intelectual y esfuerzo como equipo.
2. Las herramientas de Inteligencia Artificial fueron utilizadas exclusivamente como apoyo en tareas específicas descritas en este capítulo, no como sustituto de nuestro trabajo.
3. Todo el contenido generado por IA fue revisado, comprendido, validado y, cuando fue necesario, corregido por los integrantes del equipo.
4. Asumimos plena responsabilidad académica sobre el contenido total de este proyecto y su documentación.
5. El uso de IA se realizó conforme a las políticas de la institución educativa y los lineamientos del docente responsable.

7. Conclusiones

El desarrollo de este sistema de radar 2D demostró que es posible construir una herramienta de detección y visualización en tiempo real combinando hardware accesible como el Arduino UNO y el sensor HC-SR04 con software desarrollado en Python, resolviendo en el proceso desafíos técnicos como la sincronización entre hilos de ejecución, el manejo correcto de comunicación serial sin delimitadores de línea, y la optimización del firmware para evitar bloqueos por timeouts excesivos.

8. Bibliografía

[1] IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, IEEE Computer Society, New York, NY, USA, 1998. doi: 10.1109/IEEESTD.1998.88286

[2] ISO/IEC, *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*, ISO/IEC 25010:2011, International Organization for Standardization, Geneva, Switzerland, 2011.

[3] IEEE, *IEEE Standard for Information Technology — Systems Design — Software Design Descriptions*, IEEE Std 1016-2009, IEEE Computer Society, New York, NY, USA, 2009. doi:

10.1109/IEEESTD.2009.5167255

[4] *Python 3.x Standard Library* [docs.python.org](https://docs.python.org/3/)

[5] Tkinter (Interface Wrapper): <https://docs.python.org/3/library/tkinter.htm>

[7] ISO/IEC/IEEE, *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, ISO/IEC/IEEE 29148:2018, International Organization for Standardization, Geneva, Switzerland, 2018. doi: 10.1109/IEEESTD.2018.8559686

[8] M. Nygard, *Architecture Decision Records (ADRs)*, ThoughtWorks Technology Radar, 2017. [En línea]. Disponible en: <https://adr.github.io>